Distillation and Deployment of a Command-Parsing Language Model on a Local Cloud

Adarsh Singh Baghel

Cloud Computing Mini Project

Contents

Abstract **1**

1. Architecture and design decisions **2**
1.1 Architecture . . . 2
1.2 Technology decisions . . . 2

2. Storage, teacher labelling, and cleaning **2**
2.1 Teacher and sharding . . . 2
2.2 Cleaning results . . . 2

3. Sequence-level knowledge distillation **3**
3.1 Training configuration . . . 3
3.2 Training loss . . . 3
3.3 Validation results . . . 3
3.4 Sequence-level versus logit-based distillation . . . 4

4. Docker inference service **4**
4.1 Memory test . . . 4

5. Kubernetes deployment and reliability **4**
5.1 Deployment configuration . . . 4
5.2 Pod recovery and cold start . . . 5
5.3 Load balancing . . . 5

6. Subnetting and network access control **5**

7. CI/CD and reproducibility **6**

8. Performance, cost, and review **6**
8.1 Load-test results . . . 6
8.2 Cost estimate . . . 6
8.3 SLO observation . . . 7
8.4 Summary of limitations and improvements . . . 7

9. Conclusion **7**

Abstract

This project implements a local-cloud pipeline for converting natural-language commands into structured JSON using knowledge distillation. Phi-3.5 Mini Instruct, served through Ollama, acts as the teacher. Its generated responses are cleaned and used to fine-tune EleutherAI Pythia-70M through sequence-level distillation. The student model is packaged into a FastAPI inference service, containerized with Docker, and deployed on a local Kubernetes cluster created with k3d.

SeaweedFS provides S3-compatible object storage for datasets and model artifacts. GitHub Actions performs linting, unit testing, secret scanning, model evaluation, and a JSON-validity quality gate.

The experiment successfully demonstrates the end-to-end workflow, but the labelling stage processed 600 commands rather than the intended 3,000. Cleaning retained 513 examples, and both student versions were trained on those 513 examples. This limitation is considered when interpreting model quality.

1. Architecture and design decisions

1.1 Architecture

The implemented workflow is:

1. A command pool is stored in SeaweedFS object storage.
2. Two Kubernetes worker Pods label disjoint shards using Phi-3.5 through the Ollama API.
3. Raw teacher responses are stored in object storage.
4. A cleaning process validates JSON structure, intent values, slot values, duplicates, and dataset coverage.
5. The cleaned data is used to train Pythia-70M using sequence-level distillation on CPU.
6. Model artifacts are packaged and uploaded to SeaweedFS and published as GitHub Release assets.
7. A FastAPI service downloads the selected model from SeaweedFS at startup and exposes JSON parsing endpoints.
8. Docker packages the inference service. Kubernetes manages replicas, probes, rolling updates, ingress, and autoscaling.
9. GitHub Actions runs the automated quality checks.

1.2 Technology decisions

k3d and Kubernetes: k3d provides a lightweight local Kubernetes environment. It allowed testing Deployments, Services, Ingress, readiness and liveness probes, HPA, rolling updates, and Pod recovery without requiring a managed cloud cluster.

Indexed worker Job: workers use line-number modulo sharding. For two workers, line number n is assigned to shard n mod 2. This creates disjoint shards and allows each worker to checkpoint progress independently.

SeaweedFS: SeaweedFS exposes an S3-compatible interface. It stores datasets and model artifacts separately from the Git repository, avoiding large binary model files in source control.

CPU-only training: CPU training follows the assignment requirement and makes the training procedure reproducible on the available hardware. It is slower than accelerator-based training for larger models.

Pythia-70M: the small model is suitable for a constrained local deployment, but it is a base language model rather than a purpose-built instruction-following model. Its limited capacity contributes to imperfect structured-output accuracy.

2. Storage, teacher labelling, and cleaning

2.1 Teacher and sharding

The teacher was Phi-3.5 Mini Instruct served locally through Ollama. The generation configuration used temperature 0 and seed 42. The prompt specified eight allowed intents and the exact output keys: `intent`, `date`, `time`, and `place`.

The worker implementation assigns input line n to shard n modulo the shard count. Two workers were run in parallel, each with a limit of 300 records. Workers checkpoint their outputs every 25 records and can resume from existing checkpoint data.

The experiment processed 600 commands in total, 300 per worker.

2.2 Cleaning results

The cleaner rejected malformed JSON, wrong key sets, invalid intent values, and slot values that did not appear in the original command. Duplicate commands were removed while retaining the smallest ID. Dataset coverage was checked against the first 600 pool records.

Cleaning stage	Count
Input teacher responses	600
Invalid JSON	78
Wrong keys	0
Invalid intent	0
Invalid slot value	9
Duplicates	0
Retained training examples	513

The JSON-validity rate before cleaning was 522/600, or 87%. After all validation rules, 513 examples remained. Coverage verification reported 600 records, zero gaps, zero repeats, and zero unknown IDs.

SHA-256 of the cleaned training file:

`620dea5d8951ec84c8ec9250e7f838e71807b35ca5fa1fb1e6d5fe83e41c089ee2`

Scope deviation: the assignment target was 3,000 labelled commands. Due to the local teacher-inference time and hardware constraints, only 600 were labelled. Consequently, the student training set contained 513 examples, not the intended 1,500. This reduced dataset size is a material limitation.

3. Sequence-level knowledge distillation

3.1 Training configuration

The student model was `EleutherAI/pythia-70m`. Training used CPU execution, seed 42, learning rate 0.0002, batch size 16, maximum sequence length 128, weight decay 0.01, and warmup ratio 0.1. The training script used the teacher-generated JSON as the target sequence and masked the prompt tokens from the loss calculation.

The first 1,500 examples were permitted by the configuration, but only 513 cleaned examples were available.

3.2 Training loss

Epoch	v1 loss	v2 loss
1	0.686202	0.732607
2	0.208369	0.251726
3	0.135073	0.138523
4	0.062387	0.085232
5	0.029483	0.052558
6	—	0.017181

Both runs showed decreasing training loss. This indicates that the student learned to reproduce the training targets, but training loss alone does not establish generalization.

3.3 Validation results

The 100-command validation set was kept separate from training and used only for evaluation. The untrained Pythia baseline produced no valid JSON frames under the exact output parser.

Metric (%)	Untrained Pythia-70M	Student v1	Student v2
JSON validity	0	98	100
Intent accuracy	0	80	83
Date accuracy	0	76	75
Time accuracy	0	85	86
Place accuracy	0	81	82
Full-frame accuracy	0	45	44

v2 was selected for deployment because it achieved 100% JSON validity and the highest intent, time, and place accuracy. v1 achieved slightly better date accuracy and full-frame accuracy. The difference in full-frame accuracy is one percentage point in favour of v1.

The 100-example validation set is small, so the reported percentages should be interpreted as experimental measurements rather than precise estimates of general performance.

3.4 Sequence-level versus logit-based distillation

Sequence-level distillation trains the student using text sequences generated by the teacher. The student minimizes ordinary cross-entropy against the teacher's output tokens. In this project, the input command is paired with the teacher-generated JSON response.

Logit-based distillation instead attempts to match the teacher's probability distribution over possible next tokens, commonly using a divergence loss. This can transfer more information than a single generated sequence. However, the teacher and student here use different tokenizers and vocabularies, and the Ollama interface supplied generated text rather than teacher logits. Therefore, sequence-level distillation was used.

Important limitations include the small labelled dataset, teacher errors being copied into the student targets, and the limited instruction-following capacity of Pythia-70M.

4. Docker inference service

The inference service uses FastAPI and exposes:

- `POST /parse` for command parsing.
- `GET /healthz` for process liveness.
- `GET /readyz` for model readiness.

Responses include `X-Model-Version` and `X-Served-By` headers. The model is downloaded from the S3-compatible SeaweedFS endpoint when the service starts. The selected deployment version is v2.

The Docker image `cmd-parser:1.0` was measured at 1,253,995,980 bytes, approximately 1.25 GB.

4.1 Memory test

The standalone container was constrained to one CPU and 1 GiB of memory. Fifty sequential parsing requests were sent to the v2 model.

Metric	Result
Requests	50
HTTP 200 responses	50
Failed requests	0
Final Docker memory reading	796.4 MiB
Peak cgroup memory	1,054,191,616 bytes (approximately 1,005.5 MiB)
Memory limit	1 GiB
OOMKilled	false

The peak was approximately 98.2% of the configured memory limit. The container completed the test without an OOM kill, but the small remaining memory headroom is a deployment risk under higher concurrency or larger requests.

5. Kubernetes deployment and reliability

5.1 Deployment configuration

The application was deployed to a local k3d cluster. The Kubernetes configuration includes:

- Deployment with rolling-update strategy.
- ClusterIP Service and Traefik Ingress.
- Liveness probe on `/healthz`.

- Readiness probe on `/readyz`.
- CPU request of 500 millicores and limit of 1 CPU.
- Memory request of 512 MiB and limit of 1 GiB.
- HPA with minimum 1 replica, maximum 2 replicas, and 70% CPU target.

The current deployment was observed with one Ready replica. HPA scale-up and scale-down events were both recorded: the replica count changed from one to two when CPU exceeded the target and returned to one when utilization fell below the target.

5.2 Pod recovery and cold start

A parser Pod was deleted to test Deployment reconciliation. Kubernetes created a replacement Pod, which reached Ready state.

Three additional cold-start trials were conducted by deleting the application Pods and measuring the time from Pod creation to Ready.

Trial	Creation-to-Ready
1	5 seconds
2	5 seconds
3	5 seconds
Median	5 seconds

The model download logs showed all nine model objects were downloaded in 1.1 seconds, 0.8 seconds, and 0.8 seconds respectively. The timestamps used by the test have one-second resolution.

5.3 Load balancing

The Locust test recorded 63 requests with zero failures. Requests were served by two different replicas:

Replica	Requests served
parser replica 1	40
parser replica 2	24
Total	64

The recorded per-replica header counts sum to 64, whereas the Locust statistics report 63 requests. This discrepancy is retained rather than silently reconciled; the header log and Locust statistics may have captured slightly different request sets.

The test was manually interrupted after running longer than its nominal two-minute duration. Therefore, the measured throughput and latency represent the recorded partial run, not a complete two-minute benchmark.

Reliability limitation: the cluster ran on one laptop and one Kubernetes node. Pod recovery demonstrates workload reconciliation, not node-level or multi-machine fault tolerance.

6. Subnetting and network access control

A VLSM design was calculated from `10.20.0.0/16`, allocating the largest subnet first.

Subnet	Required hosts	CIDR	Usable hosts	Network
app	1,000	/22	1,022	10.20.0.0
public	250	/24	254	10.20.4.0
data	100	/25	126	10.20.5.0
management	20	/27	30	10.20.5.128

The allocations are contiguous and non-overlapping. The network, usable host ranges, and broadcast addresses were checked using Python's `ipaddress` module.

The public-subnet NACL denies the abusive test range `198.51.100.0/24`, permits HTTP and HTTPS, restricts SSH to the management subnet, and allows the required ephemeral return traffic. Outbound rules permit application traffic to TCP port 8000 and return traffic to client ephemeral ports.

NACLs are stateless, so request and reply packets require separate rules. Rule ordering matters because the first matching rule determines the result. The deny rule for the abusive range is placed before the web allow rules.

The packet simulator passed all six sample cases: 6/6 correct. This is a local simulator of NACL behaviour, not an actual AWS VPC deployment.

7. CI/CD and reproducibility

The GitHub Actions workflow runs:

1. Ruff linting.
2. Python unit tests.
3. Gitleaks secret scanning.
4. Student model download from the GitHub Release, checksum verification, validation prediction, and scoring.
5. A quality gate requiring at least 95% JSON validity.

The successful run was:

GitHub Actions run 42536834551

All five jobs passed. The v2 model was downloaded from the GitHub Release, and the JSON-validity quality gate passed. The project unit tests reported 21 passed and 1 skipped. Ruff and Gitleaks also passed.

Model weights are stored outside Git in SeaweedFS and as GitHub Release assets. The v2 release archive SHA-256 is:

`a948ff57c646363a5020fbfc751d4929feff17dfae814db1cd24a37ee7ca7fbf`

8. Performance, cost, and review

8.1 Load-test results

Metric	Measured value
Requests	63
Failures	0
Error rate	0%
p50 latency	4,800 ms
p95 latency	16,000 ms
Average latency	7,231.89 ms
Throughput	0.55 requests/second

The high p95 latency indicates that inference is relatively slow on the constrained local environment. The zero-failure observation applies only to this small test run.

8.2 Cost estimate

The project's rate card specifies INR 3.50 per vCPU-hour and INR 0.45 per GiB-hour. Using two replicas, each with a one-vCPU and one-GiB limit:

Hourly cost:

`2 x (1.0 x 3.50 + 1.0 x 0.45) = INR 7.90/hour`

Estimated requests per hour:

`0.55 x 3,600 = 1,980 requests/hour` (approximately)

Cost per 1,000 requests:

```
INR 7.90 / 1,980 x 1,000 = approximately INR 3.99
```

The metrics script reported INR 4.0197 per 1,000 requests using its unrounded throughput value. The report therefore uses the script's measured estimate: **INR 4.02 per 1,000 requests**.

This is a rate-card estimate, not actual cloud-provider billing. The Kubernetes environment ran locally.

8.3 SLO observation

The test used a 99% request-success target over 63 requests. Zero failures were observed. Since one failure would exceed the integer error budget of less than one request, the observed run met the target. A 63-request sample is too small to establish long-term service reliability.

8.4 Summary of limitations and improvements

Component	Evidence	Limitation	Improvement
Data and storage	600 labelled; 513 retained; coverage check passed	Below 3,000-command target	Complete labelling of the remaining pool
Model quality	v2 JSON validity 100%; full-frame 44%	Small training set and modest exact-frame accuracy	Increase clean examples and improve prompt/target quality
Latency	p50 4.8 s; p95 16 s	Slow CPU inference	Optimize generation, quantize, or use a faster inference runtime
Memory	Peak approximately 1,005.5 MiB under 1 GiB	Very little memory headroom	Reduce model/runtime memory and test concurrency
Reliability	Pod recovery and HPA scaling observed	Single-node local cluster	Test multi-node deployment and failure domains
Security and cost	Gitleaks passed; NACL simulator 6/6; INR 4.02/1,000 estimate	Simulated network and local cost model	Validate against a real cloud VPC and provider billing

9. Conclusion

The project demonstrates an end-to-end local-cloud implementation of teacher labelling, dataset cleaning, sequence-level knowledge distillation, object storage, containerized inference, Kubernetes deployment, network-rule simulation, and automated CI quality checks.

The v2 student achieved 100% valid JSON on the 100-example validation set and improved intent, time, and place accuracy over v1. However, exact full-frame accuracy remained 44%, and the experiment used only 513 cleaned training examples because only 600 commands were labelled. The deployment passed the recorded memory, cold-start, recovery, scaling, and load tests, although memory usage and latency leave room for improvement.

The results demonstrate the feasibility of the architecture on a constrained local machine, while also showing that dataset coverage, model quality, inference latency, and memory headroom are the main areas requiring further work.